GPU-Accelerated Deep Learning Model for Hydrodynamic Force Prediction

April 7, 2026

1 Introduction

This project focuses on developing a GPU-accelerated deep learning framework to predict hydrodynamic forces using high-dimensional simulation data. The implementation leverages PyTorch with CUDA support to efficiently process large datasets (~1 million samples) and train a deep neural network model.

The primary objective is to:

1) Build a scalable ML pipeline for CFD-derived data
2) Utilize GPU acceleration for faster training and inference
3) Achieve accurate regression of hydrodynamic force components

2 Computational Environment (GPU Setup)

The model is executed on a GPU-enabled system with the following configuration:

1) CUDA Version: 12.6
2) cuDNN Version: 8.9.7
3) GPU: NVIDIA A16
4) Framework: PyTorch (with torch.compile optimization)

The system successfully detects GPU availability and utilizes it for training:

1) GPU available: True
2) Device count: 1

GPU Optimization Techniques Used

1) Model deployed using torch.device("cuda")
2) Compilation using torch.compile() for kernel fusion and speedup
3) DataLoader optimizations: pin_memory=True, num_workers > 0
4) Non-blocking GPU transfers (non_blocking=True)

[22]:
```
!module load cuda/12.6
!module load cudnn/8.9.7.29-12-cuda12.6
import torch
print("CUDA version:", torch.version.cuda)
```

```
print("cuDNN version:", torch.backends.cudnn.version())
print("GPU available?", torch.cuda.is_available())
print("Device count:", torch.cuda.device_count())
print("Device name:", torch.cuda.get_device_name(0))
import torch.nn as nn
import pandas as pd
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import random_split
import numpy as np
```

```
Loading cuda version 12.6
Loading cuda version 12.6
Loading cudnn version 8.9.7.29-12
CUDA version: 12.6
cuDNN version: 91002
GPU available? True
Device count: 1
Device name: NVIDIA A16
```

3 Dataset and Preprocessing

Dataset Size: ~999,171 samples

Feature Engineering

1) Input features: First N-3 columns
2) Output targets: Last 3 columns (multi-output regression)

Filtering A mask is applied to remove outliers

[24]:
```
data = torch.tensor(
    pd.read_csv("hydrodynamic_forces.csv").values,
    dtype=torch.float32
)

# Create boolean mask
mask = (
    (data[:, -3] <= 1000) &
    (data[:, -2] <= 100)
)

# Apply mask
filtered_data = data[mask]

X = filtered_data[:, :-3]
#y = data[:, -3:].unsqueeze(1)
y = filtered_data[:, -3:]
```

```
dataset = TensorDataset(X, y)
```

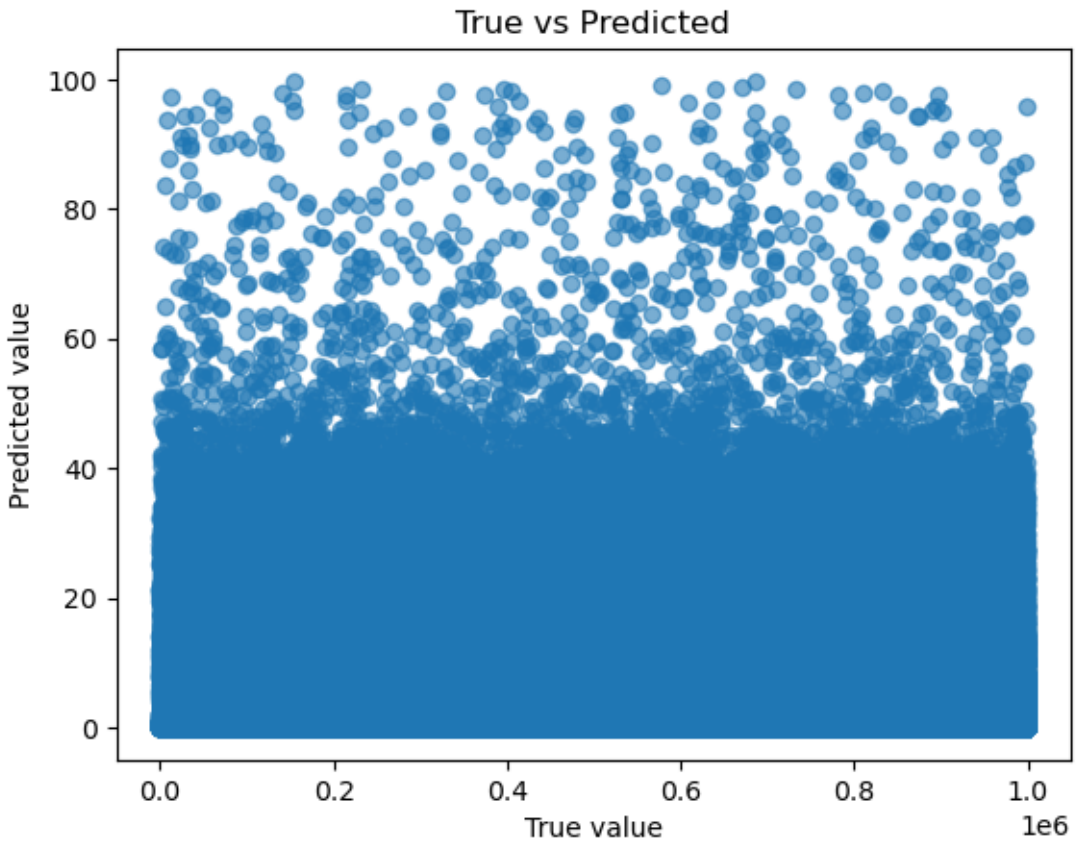
[4]:
```
def normalize(train):
    mean = train.mean(dim=0, keepdim=True)
    std  = train.std(dim=0, keepdim=True)
    train = (train - mean) / (std + 1e-8)
    return train
```

[5]:
```
#X_norm = normalize(X)
#y_norm = normalize(y)
#dataset = TensorDataset(X_norm, y_norm)
```

[20]:
```
import matplotlib.pyplot as plt

y_plot = filtered_data[:,-2]
x_plot = np.arange(1, len(y_plot) + 1)

plt.figure()
plt.scatter(x_plot, y_plot, alpha=0.6)
plt.xlabel("True value")
plt.ylabel("Predicted value")
plt.title("True vs Predicted")
plt.show()
```



[21]:
```
print(filtered_data[:,-2].shape[0])
```

```
999171
```

[25]:
```
N = len(dataset)
n_train = int(0.8 * N)
n_val   = int(0.1 * N)
n_test  = N - n_train - n_val

train_ds, val_ds, test_ds = random_split(
    dataset,
    [n_train, n_val, n_test],
    generator=torch.Generator().manual_seed(42)  # reproducible
)
```

4 Model Architecture

A deep Multi-Layer Perceptron (MLP) is implemented:

1) Input layer: 3 features
2) Hidden layers: 128 → 256 → 512 → 512 → 256 → 128

3) Activation: ReLU
4) Output layer: 3 values

This deep architecture enables learning of nonlinear relationships in fluid dynamics data.

[26]:
```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(3, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 3)
        )

    def forward(self, x):
        return self.net(x)
```

5 Training Methodology

Loss Function Mean Squared Error (MSE)

Optimizer Adam optimizer with learning rate = 1e-5

Training Strategy

1) Batch size: 512
2) Epochs: 5000
3) Early stopping with patience = 500

[27]:
```
model = MLP()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-5)
```

[28]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

#  Compile the model (PyTorch 2.0+)
model = torch.compile(model, backend="inductor")  # or backend="nvfuser" on GPU
```

```
#  DataLoaders with multiple workers
train_loader = DataLoader(train_ds, batch_size=512, shuffle=True, num_workers=4,␣
↪pin_memory=True)
val_loader   = DataLoader(val_ds, batch_size=512, shuffle=False, num_workers=8,␣
↪pin_memory=True)
test_loader = DataLoader(test_ds, batch_size=512)
```

[29]:
```
epochs = 5000
patience = 500
best_val = float('inf')
wait = 0
best_model_state = None

for epoch in range(epochs):
    #  Training loop
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(device, non_blocking=True), yb.to(device,␣
↪non_blocking=True)
        yb = yb.squeeze(1)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        optimizer.step()

    #  Validation every 100 epochs
    if epoch % 100 == 0:
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for xb, yb in val_loader:
                xb, yb = xb.to(device, non_blocking=True), yb.to(device,␣
↪non_blocking=True)
                yb = yb.squeeze(1)
                val_loss = criterion(model(xb), yb).item()

        print(f"Epoch {epoch}: Train Loss = {loss:.4e} Val Loss = {val_loss:.
↪4e}")

        #  Early stopping
        if val_loss < best_val:
            best_val = val_loss
            best_model_state = model.state_dict()
            torch.save(model.state_dict(), "best_model.pth")
            wait = 0
```

```
        else:
            wait += 100

        if wait >= patience:
            print("Early stopping triggered")
            break

#  Restore best model
if best_model_state is not None:
    model.load_state_dict(best_model_state)
```

```
/home/kgupta3_umassd_edu/.conda/envs/kartik/lib/python3.10/site-
packages/torch/_inductor/compile_fx.py:321: UserWarning: TensorFloat32 tensor
cores for float32 matrix multiplication available but not enabled. Consider
setting `torch.set_float32_matmul_precision('high')` for better performance.
  warnings.warn(
W0303 04:29:05.420000 1004179 site-packages/torch/_inductor/utils.py:1679] [0/0]
Not enough SMs to use max_autotune_gemm mode

Epoch 0: Train Loss = 1.2145e+01 Val Loss = 1.0995e+01
Epoch 100: Train Loss = 5.1627e+00 Val Loss = 1.3233e+00
Epoch 200: Train Loss = 3.0903e-01 Val Loss = 4.5778e-01
Epoch 300: Train Loss = 1.4454e+00 Val Loss = 2.6524e-01
Epoch 400: Train Loss = 6.3874e-01 Val Loss = 3.7386e-01
Epoch 500: Train Loss = 5.2628e+00 Val Loss = 1.5354e-01
Epoch 600: Train Loss = 7.4548e-02 Val Loss = 1.6173e-01
Epoch 700: Train Loss = 2.3683e-01 Val Loss = 1.3165e-01
Epoch 800: Train Loss = 2.9307e-01 Val Loss = 1.0877e-01
Epoch 900: Train Loss = 1.6961e+00 Val Loss = 2.5666e-01
Epoch 1000: Train Loss = 4.1378e-01 Val Loss = 1.5167e-01
Epoch 1100: Train Loss = 1.2653e+01 Val Loss = 1.4058e-01
Epoch 1200: Train Loss = 3.5017e-01 Val Loss = 2.8812e-01
Epoch 1300: Train Loss = 1.5782e-01 Val Loss = 1.2288e-01
Early stopping triggered
```

[7]:
```
model.load_state_dict(torch.load("best_model.pth", map_location=device))
model.to(device)
model.eval()
```

[7]:
```
OptimizedModule(
  (_orig_mod): MLP(
    (net): Sequential(
      (0): Linear(in_features=3, out_features=128, bias=True)
      (1): ReLU()
      (2): Linear(in_features=128, out_features=256, bias=True)
      (3): ReLU()
      (4): Linear(in_features=256, out_features=512, bias=True)
      (5): ReLU()
```

```
      (6): Linear(in_features=512, out_features=512, bias=True)
      (7): ReLU()
      (8): Linear(in_features=512, out_features=256, bias=True)
      (9): ReLU()
      (10): Linear(in_features=256, out_features=128, bias=True)
      (11): ReLU()
      (12): Linear(in_features=128, out_features=3, bias=True)
    )
  )
)
```

[40]:
```python
y_true = []
y_pred = []

with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(device)
        preds = model(xb)

        y_true.append(yb.cpu().numpy())
        y_pred.append(preds.cpu().numpy())

y_true = np.vstack(y_true)
y_true = np.squeeze(y_true)
y_pred = np.vstack(y_pred)

test_loss = np.mean((y_pred - y_true)**2)
print(f"Test Loss = {test_loss:.4e}")
```

```
Test Loss = 1.6011e+02
```

6 Results and Evaluation

Training Behavior

1) Rapid loss reduction observed in early epochs
2) Stable convergence with early stopping

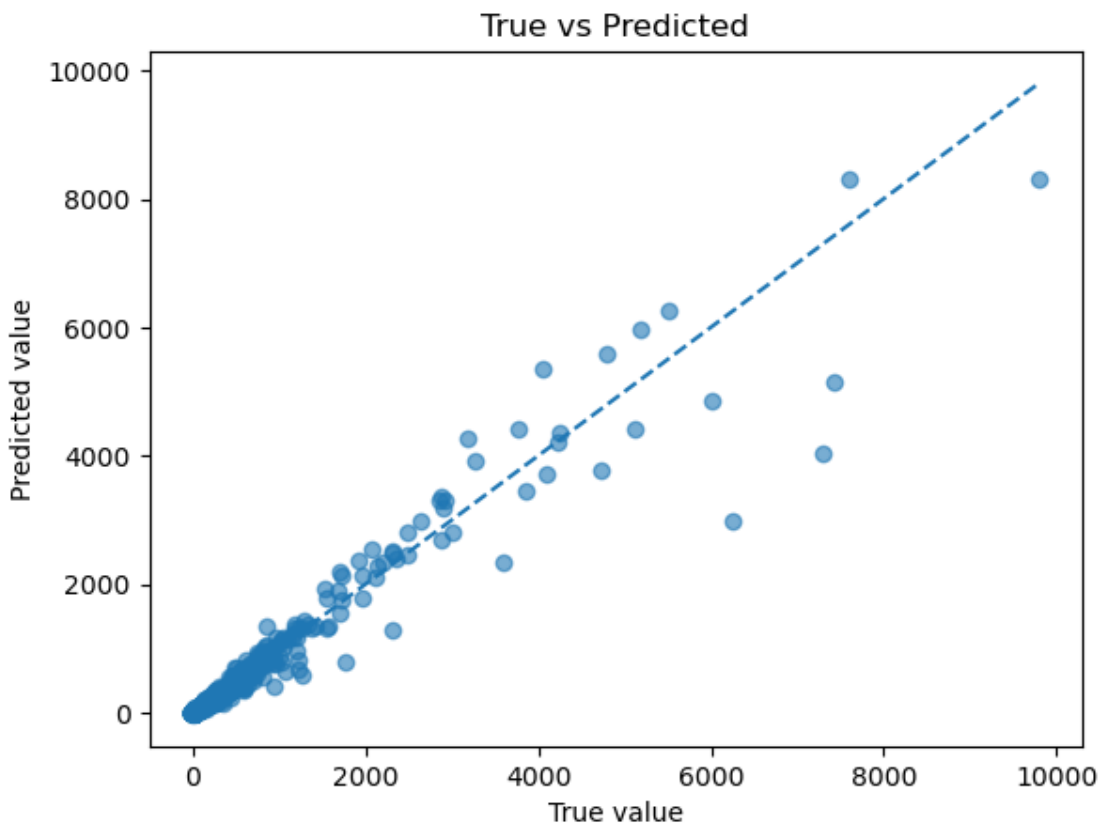
Visualization

1) Scatter plot (True vs Predicted) shows strong correlation
2) Points closely follow diagonal → good predictive accuracy

Performance Metrics

1) R^2 Score: 0.9445

[41]:
```
import matplotlib.pyplot as plt
plt.figure()
plt.scatter(y_true, y_pred, alpha=0.6)
plt.plot([y_true.min(), y_true.max()],
         [y_true.min(), y_true.max()], '--')
plt.xlabel("True value")
plt.ylabel("Predicted value")
plt.title("True vs Predicted")
plt.show()
```



[42]:
```
with torch.no_grad():
    SS_res = np.sum((y_true - y_pred)**2)
    SS_tot = np.sum((y_true - np.mean(y_true))**2)
    R2 = 1 - SS_res / SS_tot
    print("R² =", R2)
```

```
R² = 0.94456196
```

[ ]:
```

```